

Relatório de Avaliação de Desempenho: Arquiteturas Cliente-Servidor e Peer-to-Peer (P2P)

Universidade Federal de Sergipe (UFS)

Disciplina: Sistemas Distribuídos

Atividade: Atividade 01 - Unidade 2

Autor: Brenno Phelipe Silva dos Santos - 202400050750

1 Introdução e Metodologia

Este relatório apresenta a avaliação de desempenho na transferência de arquivos em rede, comparando abordagens centralizadas (Cliente-Servidor) e descentralizadas (Peer-to-Peer - P2P). O objetivo é analisar o comportamento do sistema sob carga concorrente, registrando os tempos mínimo, médio e máximo de conclusão para diferentes tamanhos de arquivo e arquiteturas.

Os testes foram conduzidos simulando 10 nós clientes simultâneos realizando o download de arquivos gerados em memória com tamanhos de 5 MB, 50 MB e 500 MB. Para evitar gargalos de I/O de disco local que pudessem mascarar o desempenho da rede, os dados recebidos pelos clientes foram imediatamente descartados em memória.

Foram testadas quatro variações arquiteturais:

1. **Cliente-Servidor (Modo 1):** Atendimento sequencial estrito (1 cliente por vez).
2. **Cliente-Servidor (Modo 2):** Atendimento simultâneo irrestrito (1 thread por cliente).
3. **Cliente-Servidor (Modo 3):** Atendimento com concorrência limitada (Thread pool de 3 clientes).
4. **Peer-to-Peer (P2P):** Transferência baseada no protocolo BitTorrent utilizando o cliente de terminal `aria2c`.

2 Resultados Obtidos

Os testes foram consolidados com base em 10 execuções simultâneas por cenário.

Tabela 1: Desempenho com Arquivo de 5 MB

Arquitetura	Variação	Mínimo (ms)	Médio (ms)	Máximo (ms)
Cliente-Servidor	Modo 1: Sequencial	29	42.70	56
Cliente-Servidor	Modo 2: Multi-thread	39	42.30	48
Cliente-Servidor	Modo 3: Thread Pool (3)	34	40.20	47

Tabela 2: Desempenho com Arquivo de 50 MB

Arquitetura	Variação	Mínimo (ms)	Médio (ms)	Máximo (ms)
Cliente-Servidor	Modo 1: Sequencial	57	155.50	236
Cliente-Servidor	Modo 2: Multi-thread	104	147.80	171
Cliente-Servidor	Modo 3: Thread Pool (3)	74	138.30	191
Peer-to-Peer (P2P)	BitTorrent	1035	1053.70	1063

3 Análise e Discussão de Desempenho

A análise cruzada das arquiteturas revela como a gestão de concorrência e a distribuição de banda afetam diretamente a experiência do utilizador e a carga sobre a infraestrutura.

3.1 Arquitetura Cliente-Servidor: Modo 1 (Sequencial)

- **Comportamento:** O servidor bloqueia a conexão e atende exclusivamente um cliente até o fim, formando uma fila de espera para os demais.
- **Justificativa dos Tempos:** Apresentou invariavelmente o **menor Tempo Mínimo** (ex: 57 ms nos 50 MB e 272 ms nos 500 MB). Isso ocorre porque o primeiro cliente da fila detém 100% dos recursos do servidor (I/O, CPU e banda de rede) sem qualquer contenção. No entanto, apresentou o **maior Tempo Máximo e a pior Média** global, pois o 10^o cliente sofreu a penalidade de aguardar ocioso na fila o término das 9 transferências anteriores (alcançando 2478 ms na carga de 500 MB). É uma arquitetura altamente ineficiente para múltiplos nós.

3.2 Arquitetura Cliente-Servidor: Modo 2 (Multi-thread Livre)

- **Comportamento:** O servidor aceita todas as conexões simultaneamente, instanciando uma nova Thread para cada cliente, dividindo imediatamente os recursos.
- **Justificativa dos Tempos:** O **Tempo Mínimo eleva-se** (ex: salta para 832 ms nos 500 MB) porque o primeiro cliente já não tem recursos dedicados; a banda e o processador são fatiados por 10 desde o primeiro milissegundo. Contudo, o **Tempo Máximo cai drasticamente** em relação ao Modo 1 (de 2478 ms para 1252 ms), pois elimina-se a fila de inatividade. O problema desta abordagem é visível nos 500 MB: a criação descontrolada de threads gera *overhead* de troca de contexto (*Context Switching*) no Sistema Operativo do servidor, penalizando o tempo médio geral.

Tabela 3: Desempenho com Arquivo de 500 MB

Arquitetura	Variação	Mínimo (ms)	Médio (ms)	Máximo (ms)
Cliente-Servidor	Modo 1: Sequencial	272	1376.00	2478
Cliente-Servidor	Modo 2: Multi-thread	832	1029.00	1252
Cliente-Servidor	Modo 3: Thread Pool (3)	402	852.70	1275

3.3 Arquitetura Cliente-Servidor: Modo 3 (Thread Pool)

- **Comportamento:** O servidor utiliza uma fila controlada e limita a concorrência a um número saudável de threads ativas (neste teste, 3 simultâneas).
- **Justificativa dos Tempos:** Entregou de forma consistente o **melhor Tempo Médio** em todas as volumetrias (40.20 ms, 138.30 ms e 852.70 ms). Esta arquitetura combina as vantagens dos modelos anteriores: ao limitar a contenção a 3 transferências simultâneas, evita a saturação de I/O e a sobrecarga de troca de contexto do Modo 2, enquanto acelera dramaticamente o esvaziamento da fila de espera em comparação ao Modo 1. Representa o ponto de equilíbrio de eficiência em arquiteturas centralizadas.

3.4 Arquitetura Peer-to-Peer (BitTorrent)

- **Comportamento:** A transferência é descentralizada. O nó central (Seeder) envia fragmentos, e os clientes (Leechers) passam a transmitir os fragmentos recebidos uns aos outros.
- **Justificativa dos Tempos:** Nos testes de 50 MB em ambiente local (onde a rede física é o mesmo host), o P2P apresentou um tempo absoluto superior (Mínimo de 1035 ms). Isso justifica-se pelo **elevado *overhead* do protocolo**: o BitTorrent necessita realizar consultas ao *tracker*, estabelecer ligações TCP (*Handshakes*) múltiplamente entre os nós, trocar *bitmaps* de peças e processar hashes SHA-1 para garantir a integridade de cada fragmento. Para ficheiros pequenos numa rede local ultrarrápida, o custo matemático e de negociação de rede do protocolo demora mais do que a própria transferência *raw* do TCP.
- **Escalabilidade Comprovada:** O grande destaque do P2P não é a sua velocidade isolada, mas a sua **curva de tempo plana**. A diferença entre o primeiro cliente a terminar e o último foi de apenas 28 milissegundos (1035 ms vs 1063 ms). Ao contrário do modelo Cliente-Servidor, onde o tempo máximo degrada linearmente com a adição de clientes, o modelo P2P escala perfeitamente. Se o teste envolvesse 1.000 clientes, os servidores centralizados entrariam em colapso, enquanto no P2P, como cada novo cliente atua simultaneamente como servidor (*uploading*), a carga é diluída, mantendo a coesão temporal independentemente da demanda.